

Lots of word vomit here but skip to the end for more detailed explanation of each function and final graphs!

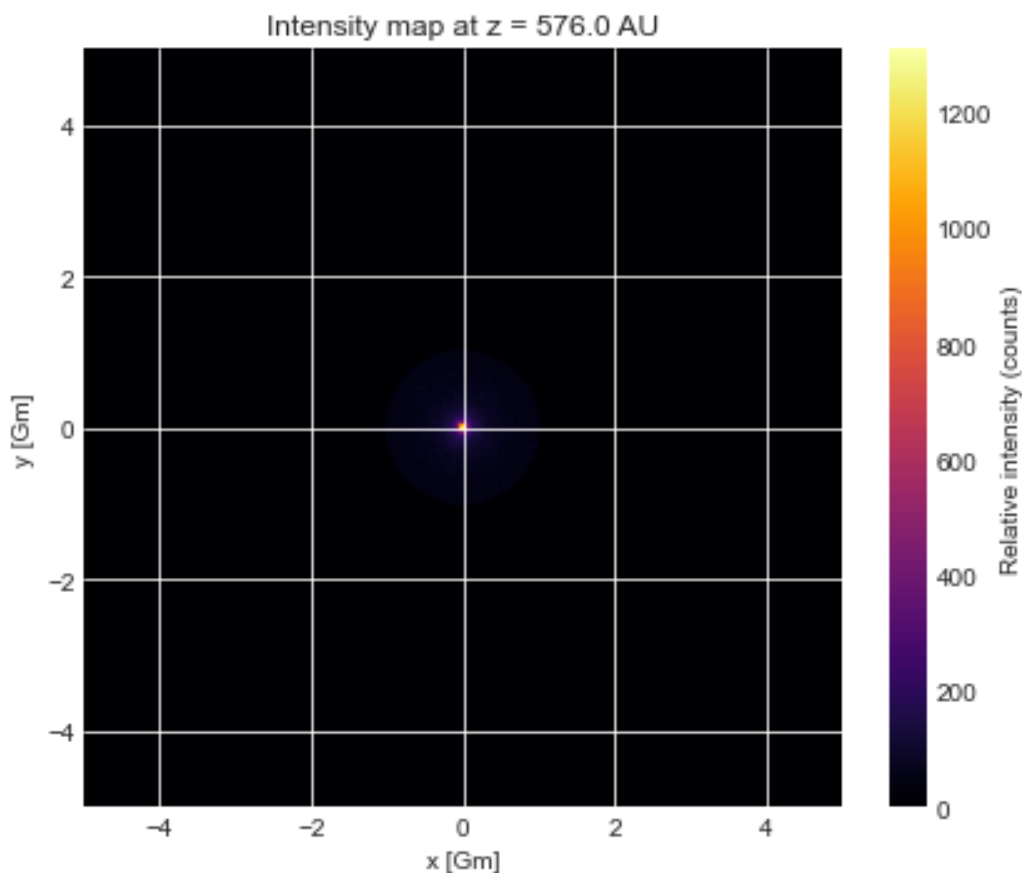
The code I put on uses Newton-Raphson to come up with an optimised focal length (i.e. how far away the receiver should ideally be) for the Sun, for the highest intensity. So this is specifically for where we should put a probe to receive signals from far away I think! I have plotted the history of how the NR code developed to show how it reached the result.

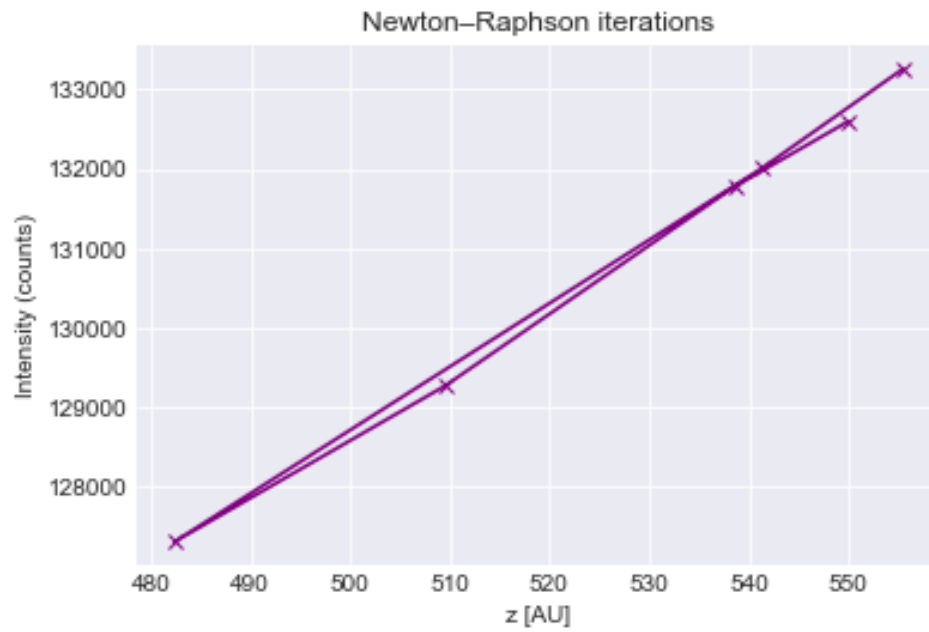
Monte-Carlo is also used to simulate a high number of random rays coming in to like get the intensity profile that we would receive, and have also coded something close to Emily's to get an intensity profile picture.

Runs can come up with quite different results (bc monte carlo can give such varied results unless u run it enough for my laptop to explode... that or the code doesn't work BUT I THINK IT DOES) but from online we know it should be around 550 AU and it does get close to that – we only need An attempt that gets us close to that for the poster anyways!

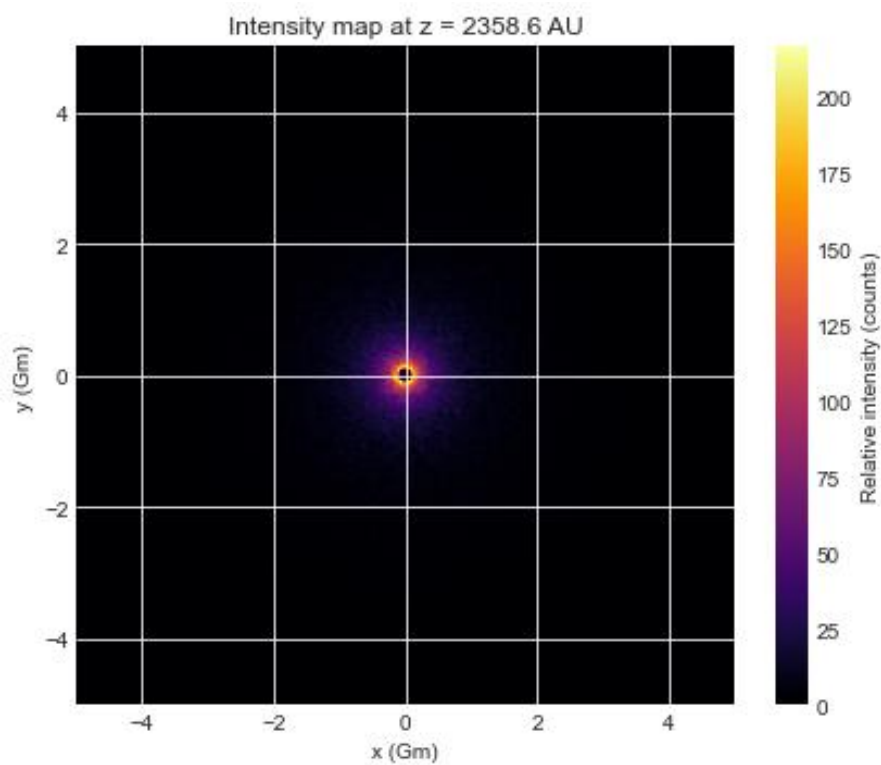
I think it works? Required a lot of inspiration from other code online but at least to me the logic checks out ... 'Counts' refers to the amount of simulated beams hitting the receiver – not anything of value irl but I guess proportional to power or amplitude of the signal(?)

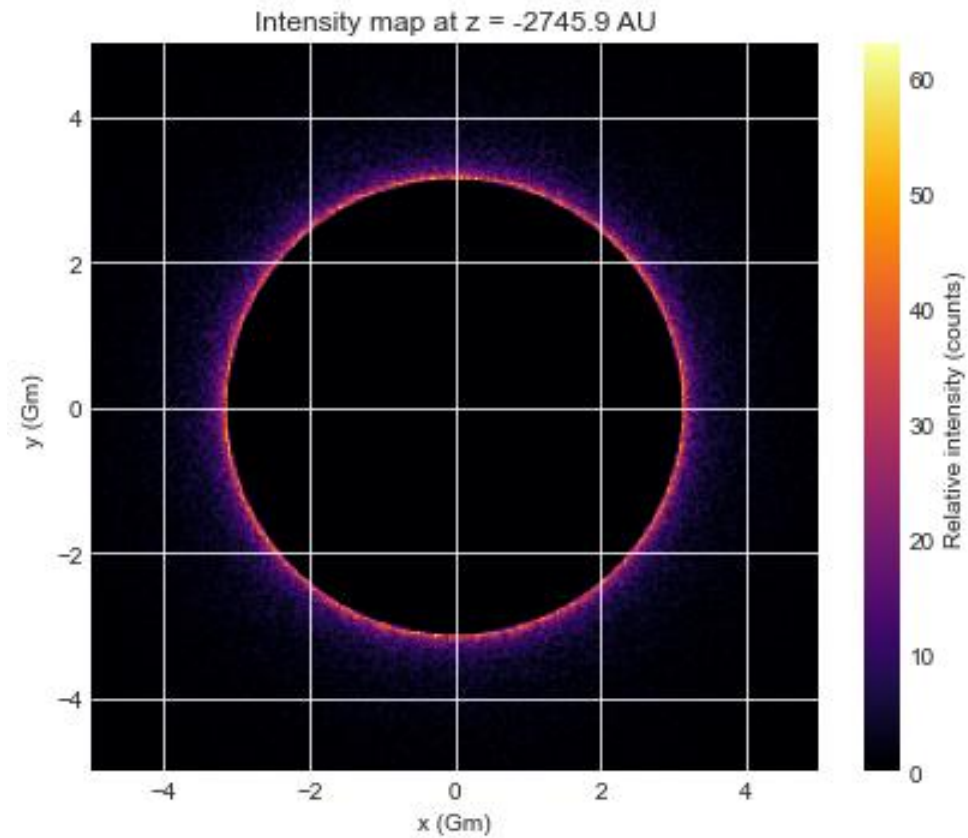
This is an example where it got 576AU (pretty close) :





You get some cool shapes for focal distances where the code has bugged out (too high, negative etc) also:

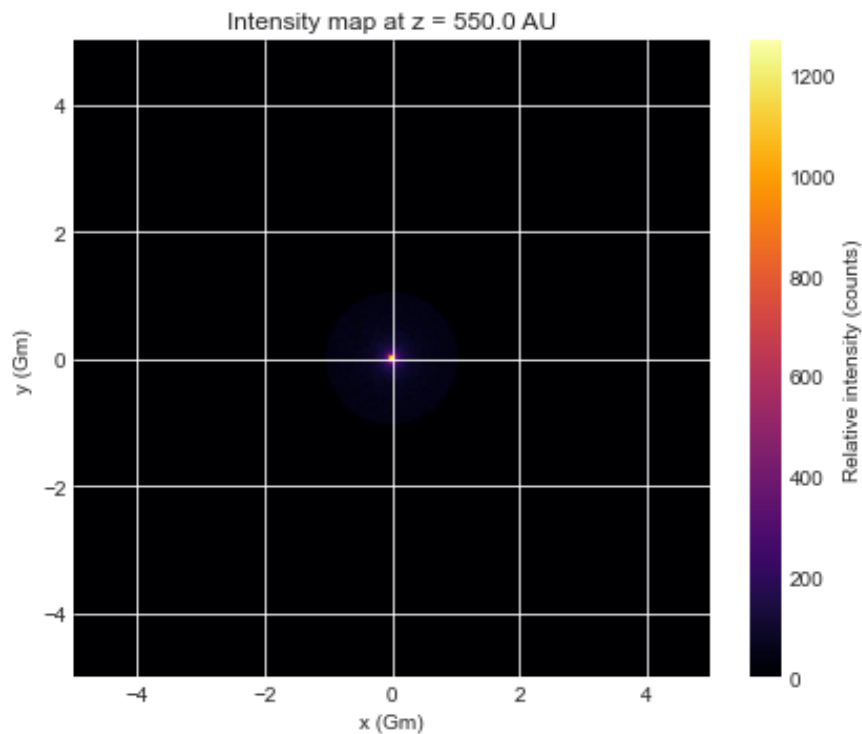




Negative one maybe interesting as negative focal length means image is behind lens hence the black circle would be the blocking from the sun / the object or something idk?

ALSO. If unconvinced with the Newton-Raphson stuff, if we just used 550 AU (justifying with either a reference or using other arguments) and ran the plotting code, that still uses Monte-Carlo!

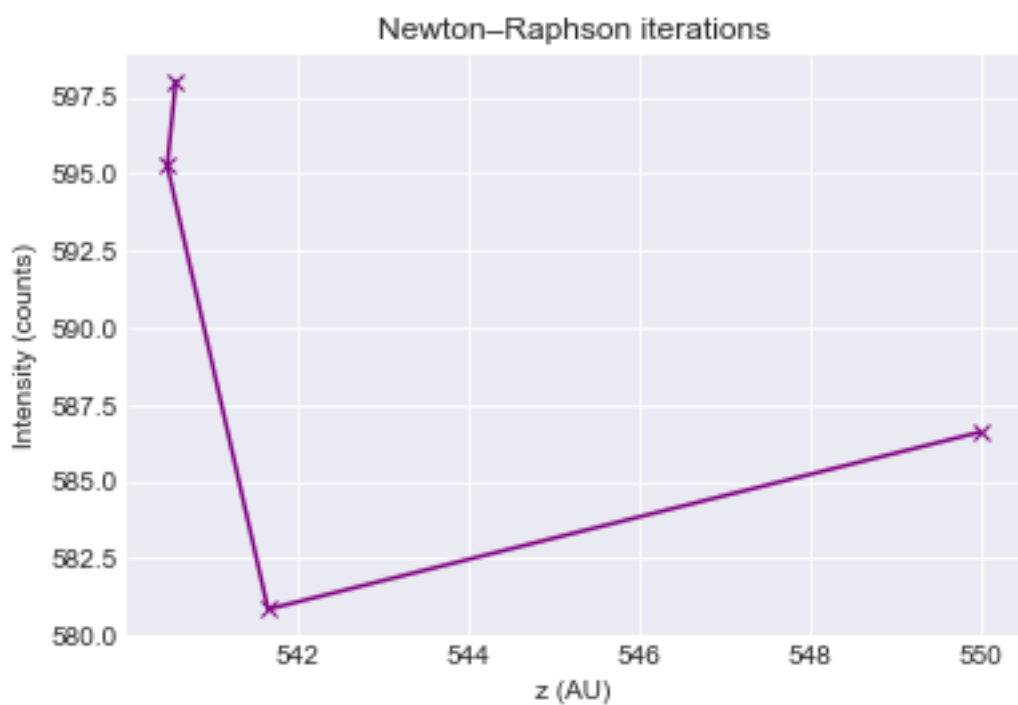
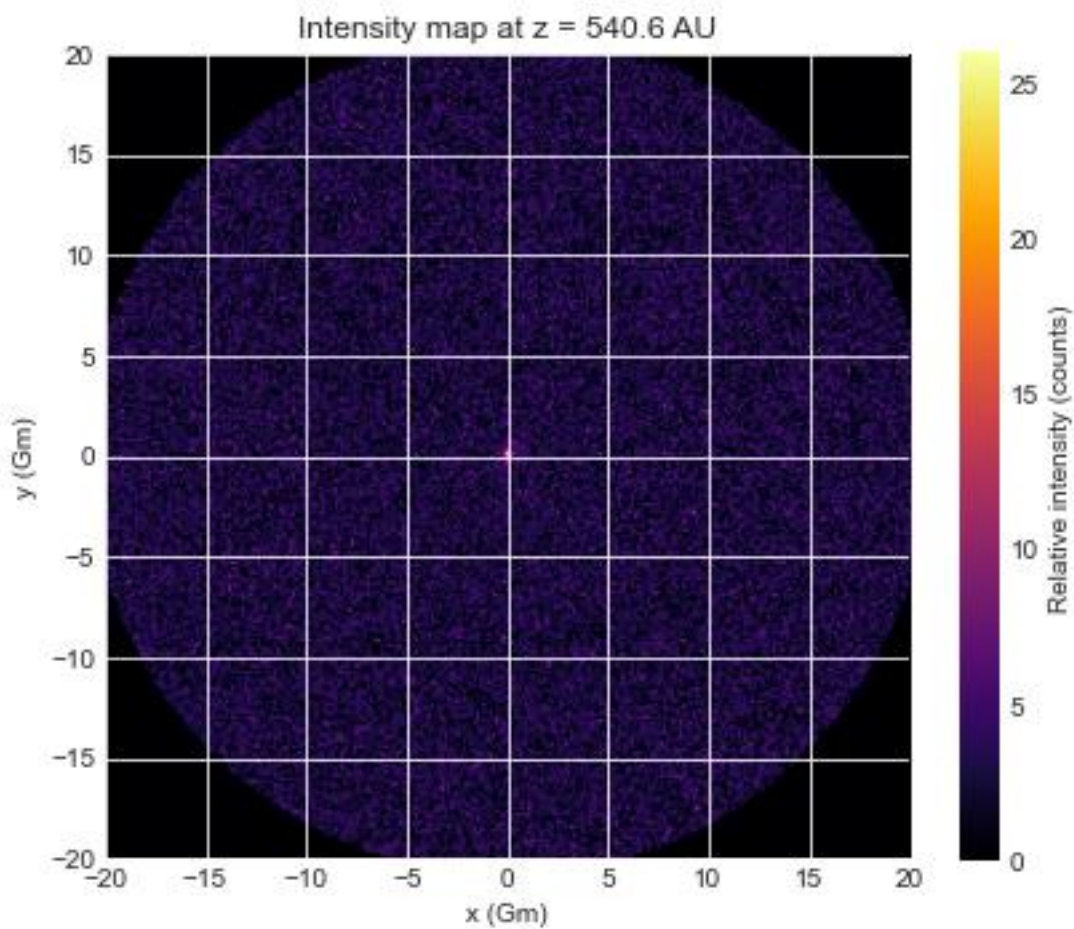
This is 550:

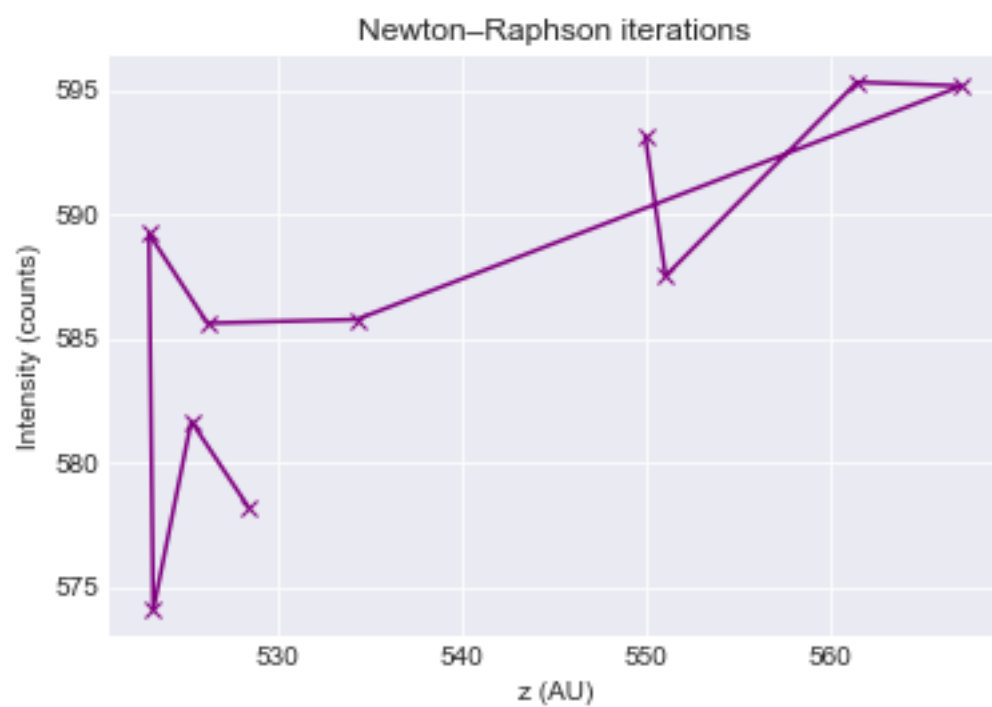
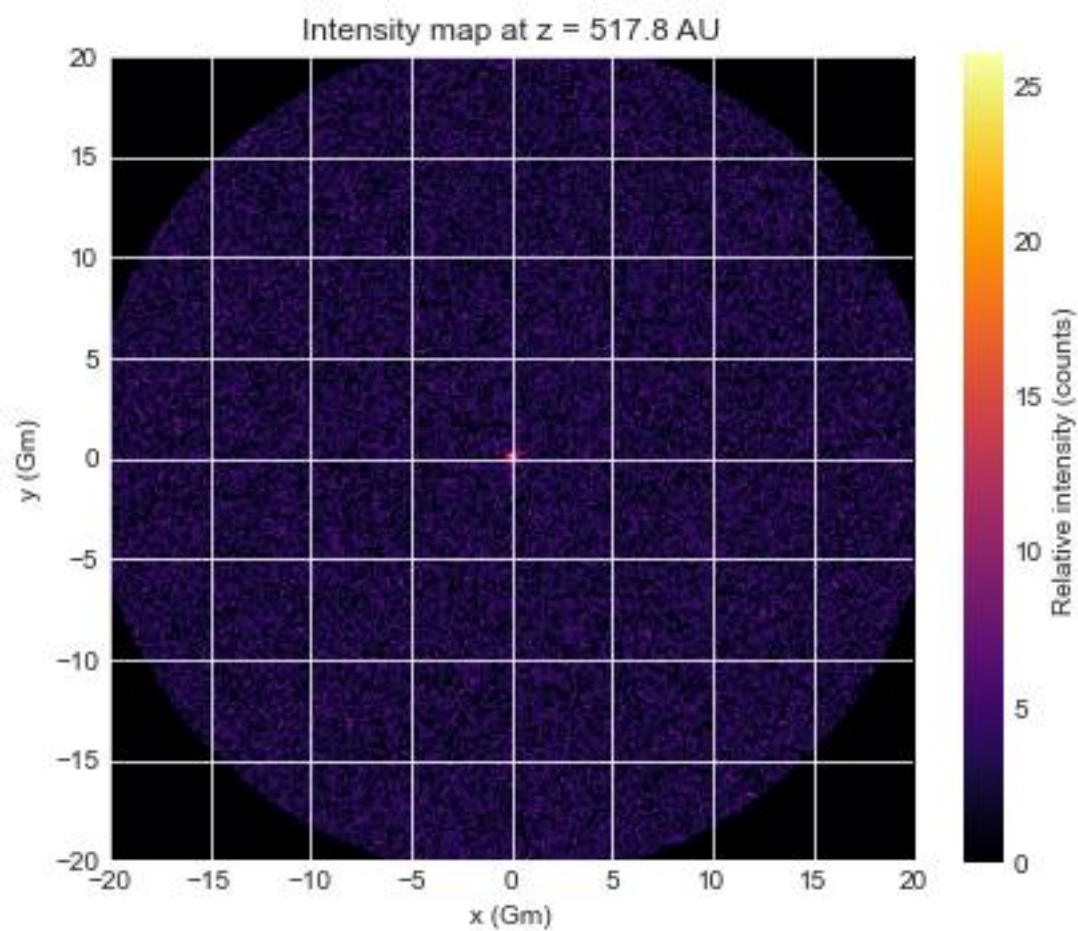


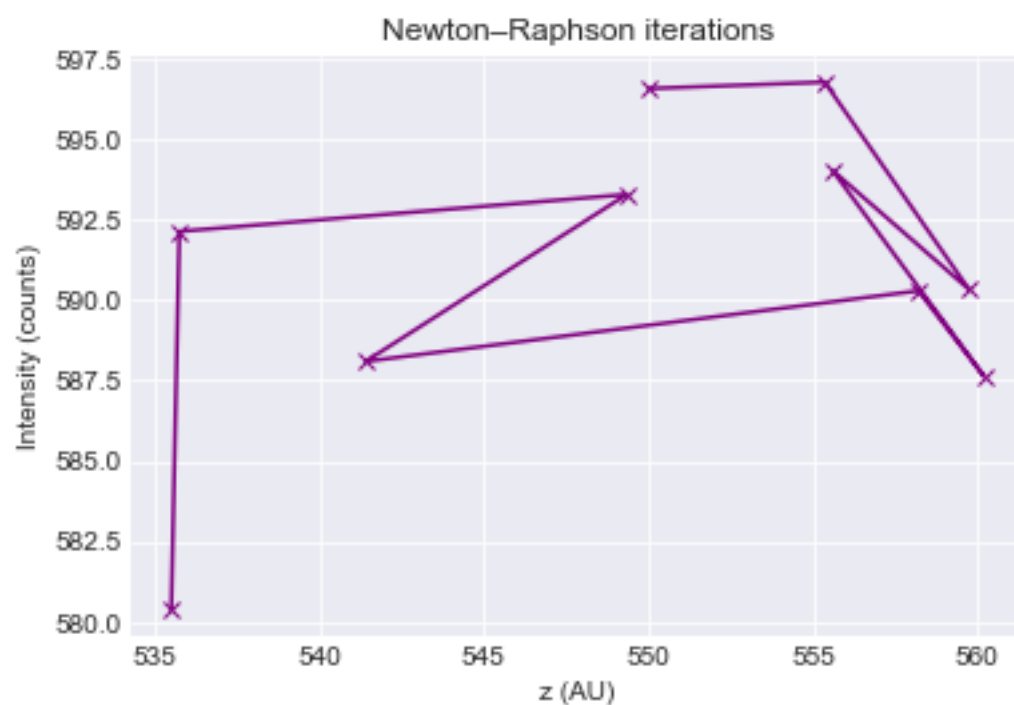
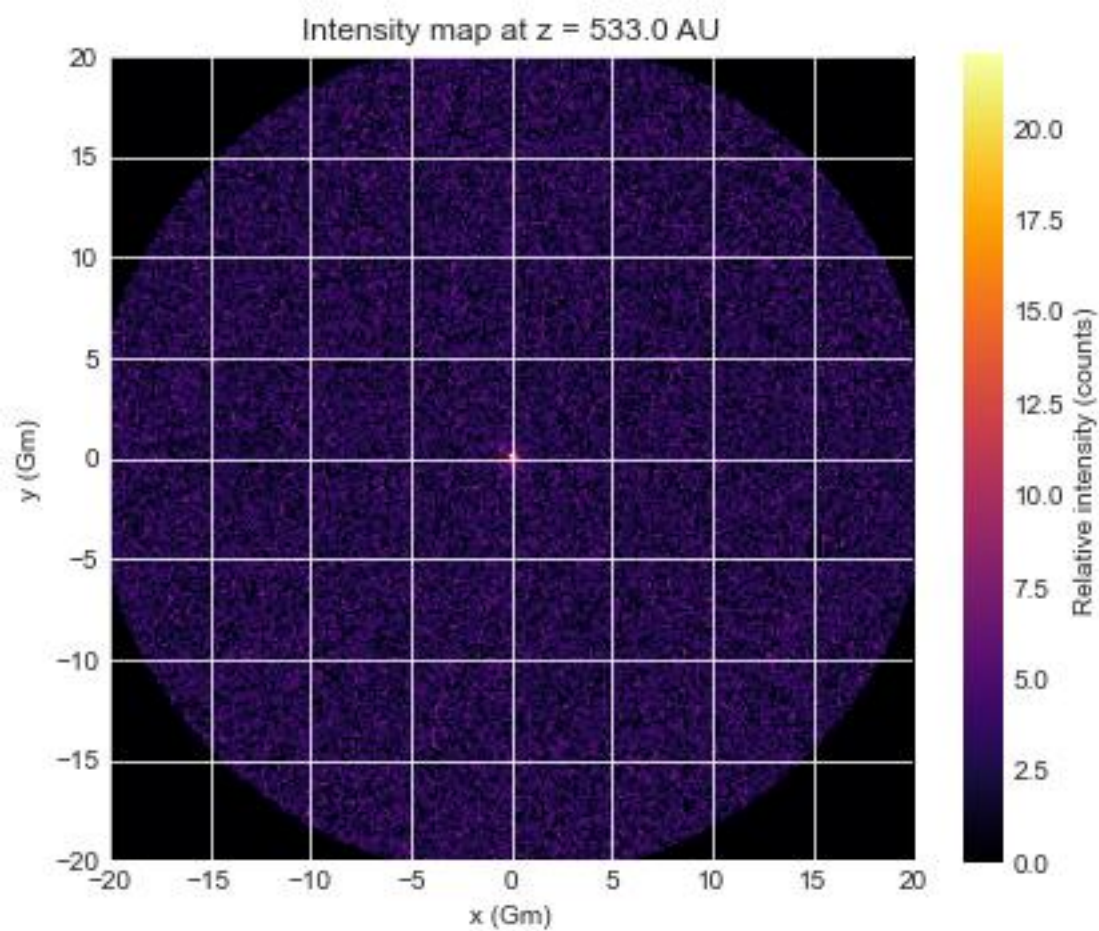
UPDATE

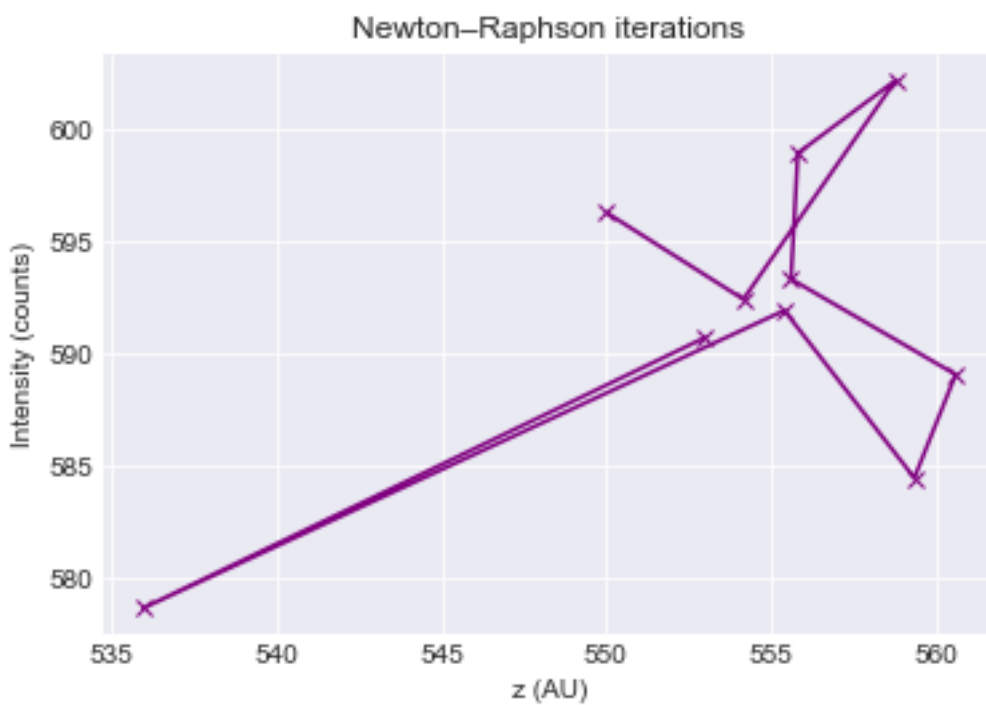
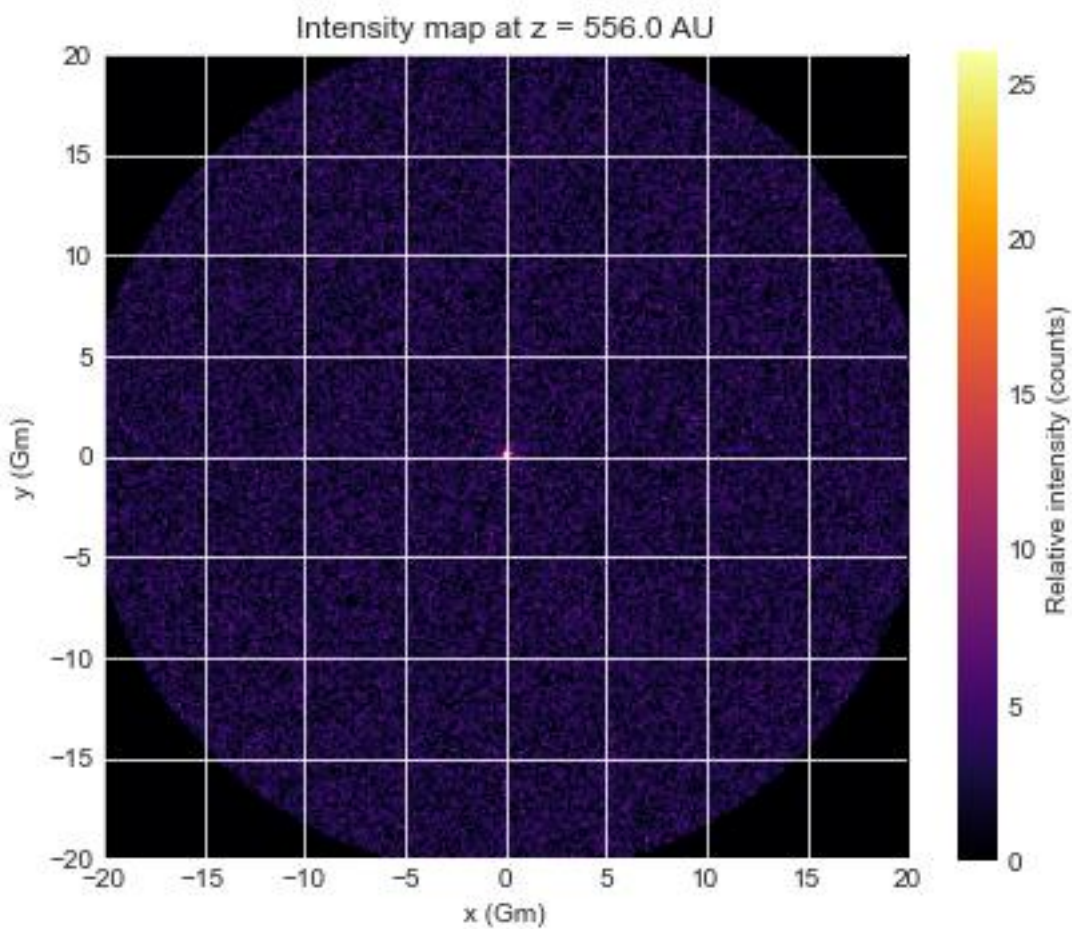
Have worked out a way to make it more consistent by increasing the impact parameter maximum (ends up w more noise but still w focussed spot in middle), and then calculating an average of the intensities over 20 attempts, and increasing the size of the axes in the intensity map.

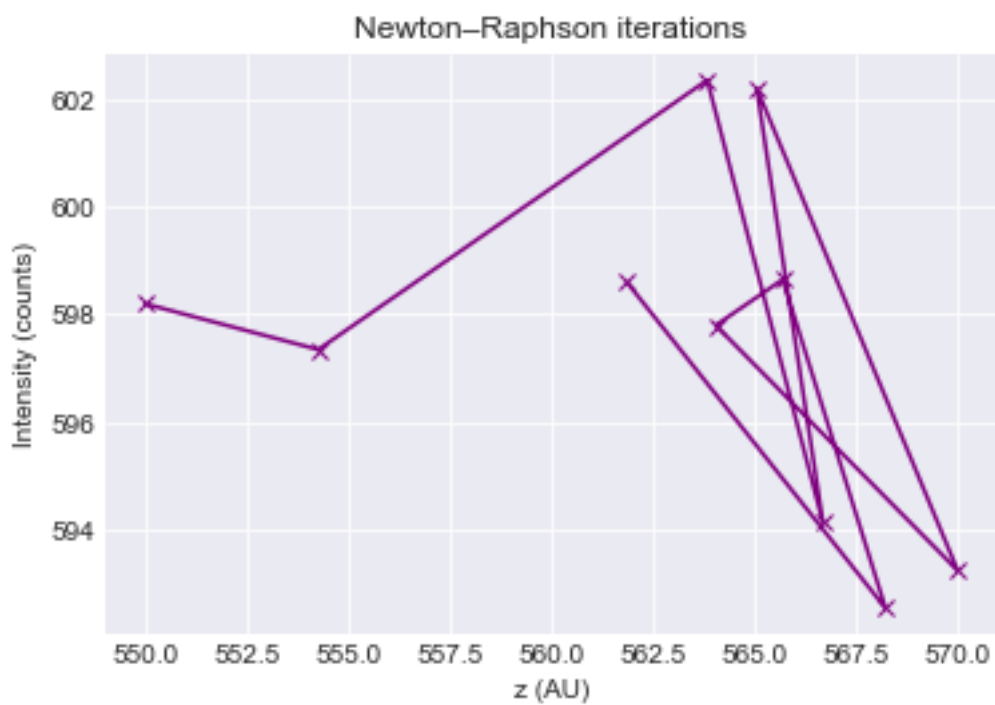
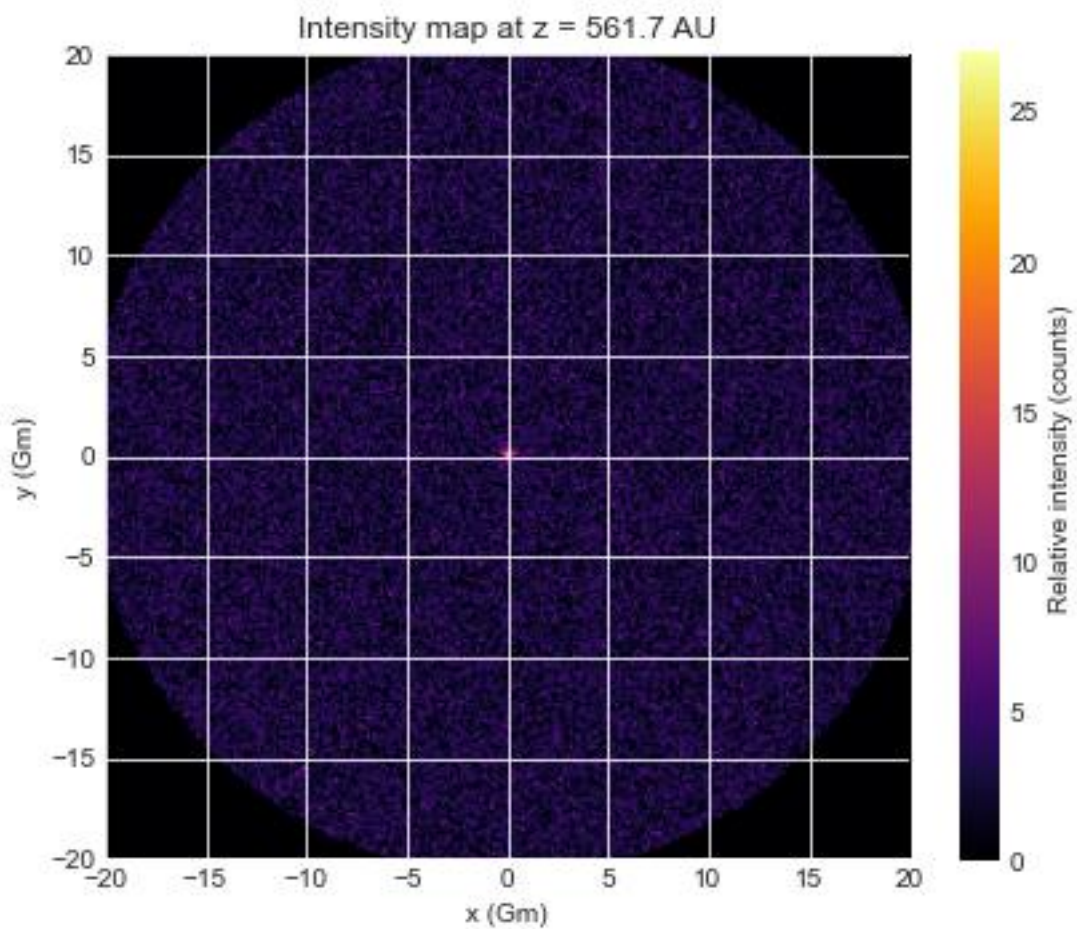
5 diff runs of the code w impact param as $30 * R_{\text{sun}}$ are all consistent all close to whats expected:











BASICALLY

- minimum distance that the receiver (z minimum) can be is 550AU because (from NASA powerpoint)

The gravitational deflection angle of light passing a massive body is

- $\theta = (4GM/c^2)(1/r) \quad (1)$

where r is the distance by which the rays being focused miss the center of the sun. The distance F to the gravitational focus is:

- $F = (4GM/c^2)^{-1} r^2 \quad (2)$

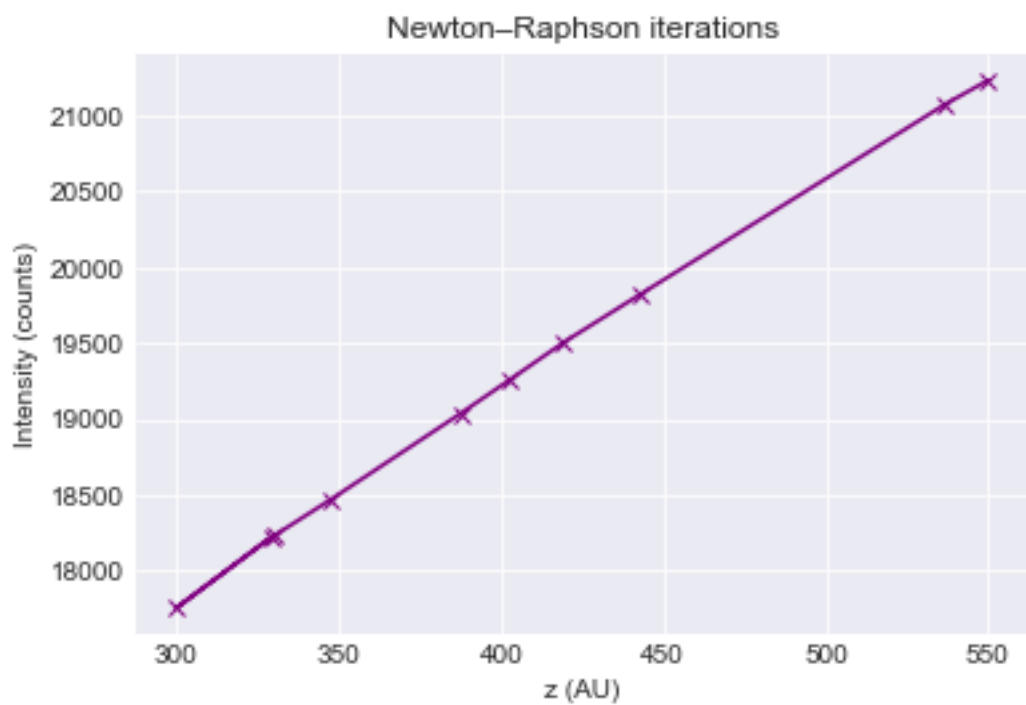
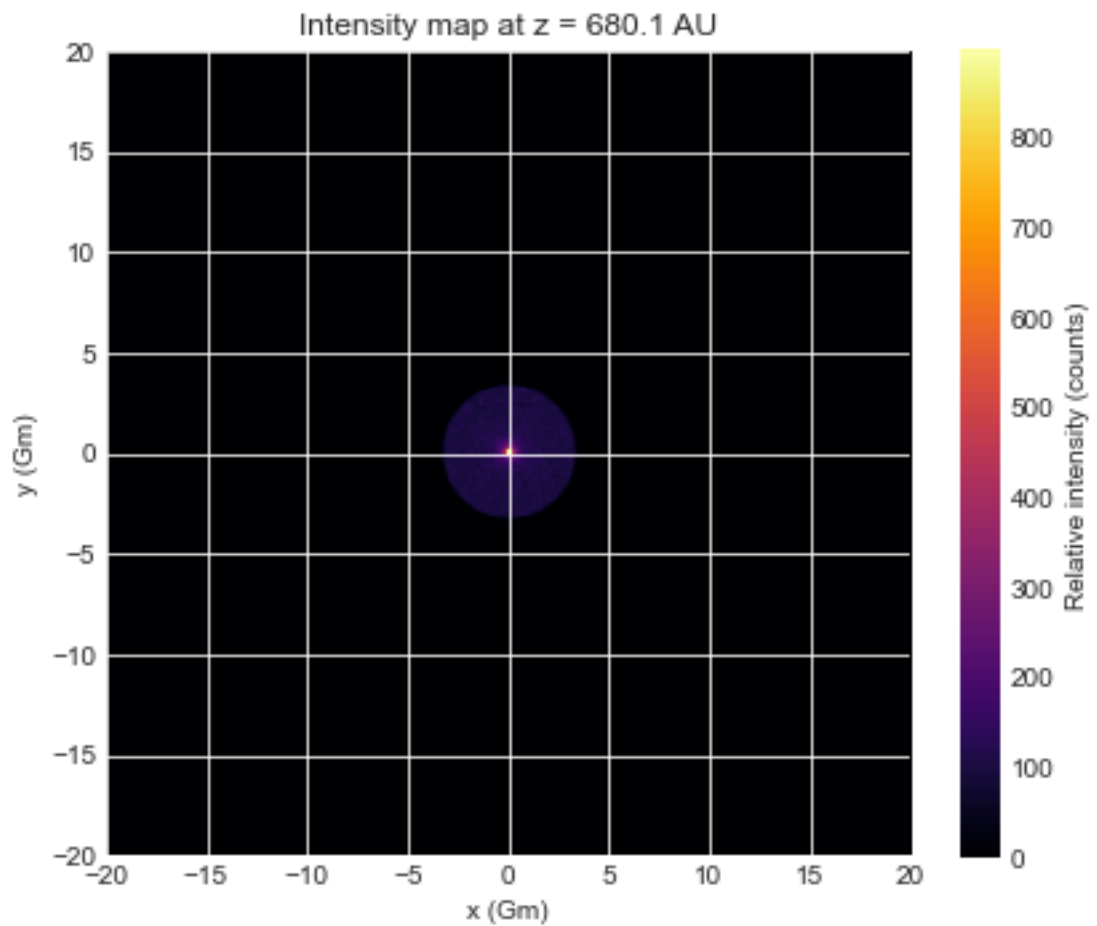
The condition required to use of the gravitational lens as a telescope is that the light from the distant target reaching the focus must not intersect the surface of the sun, and hence r must be greater than the radius of the sun (about 700,000 km).

Inserting $r = R_{\text{sun}}$ defines the minimum gravitational focus.

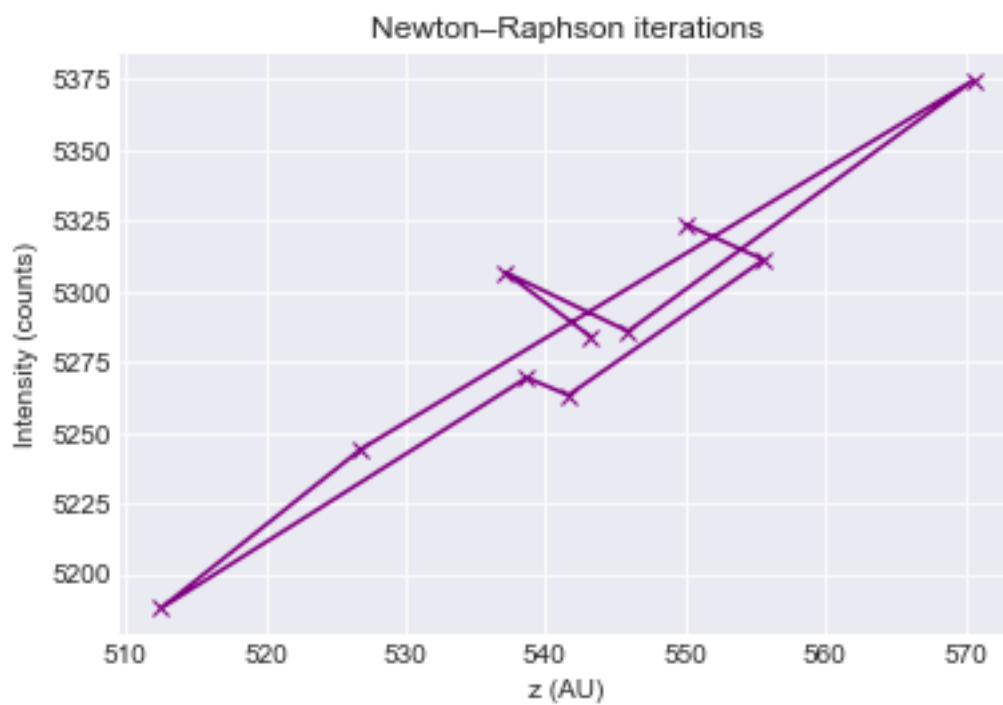
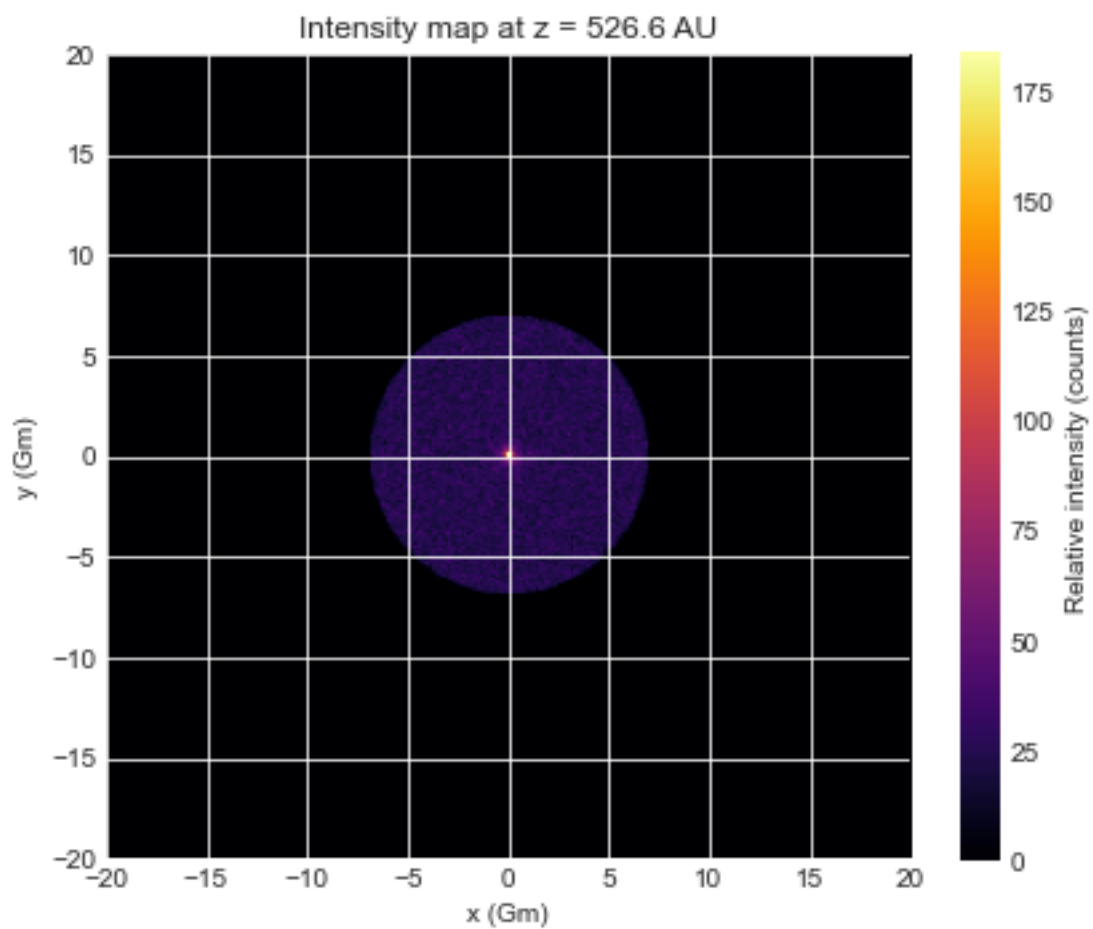
For the sun, this minimum focal distance (at which the focussed rays just skim the surface of the sun) is ~550 AU

--any distance beyond this will **also** be on the focal line

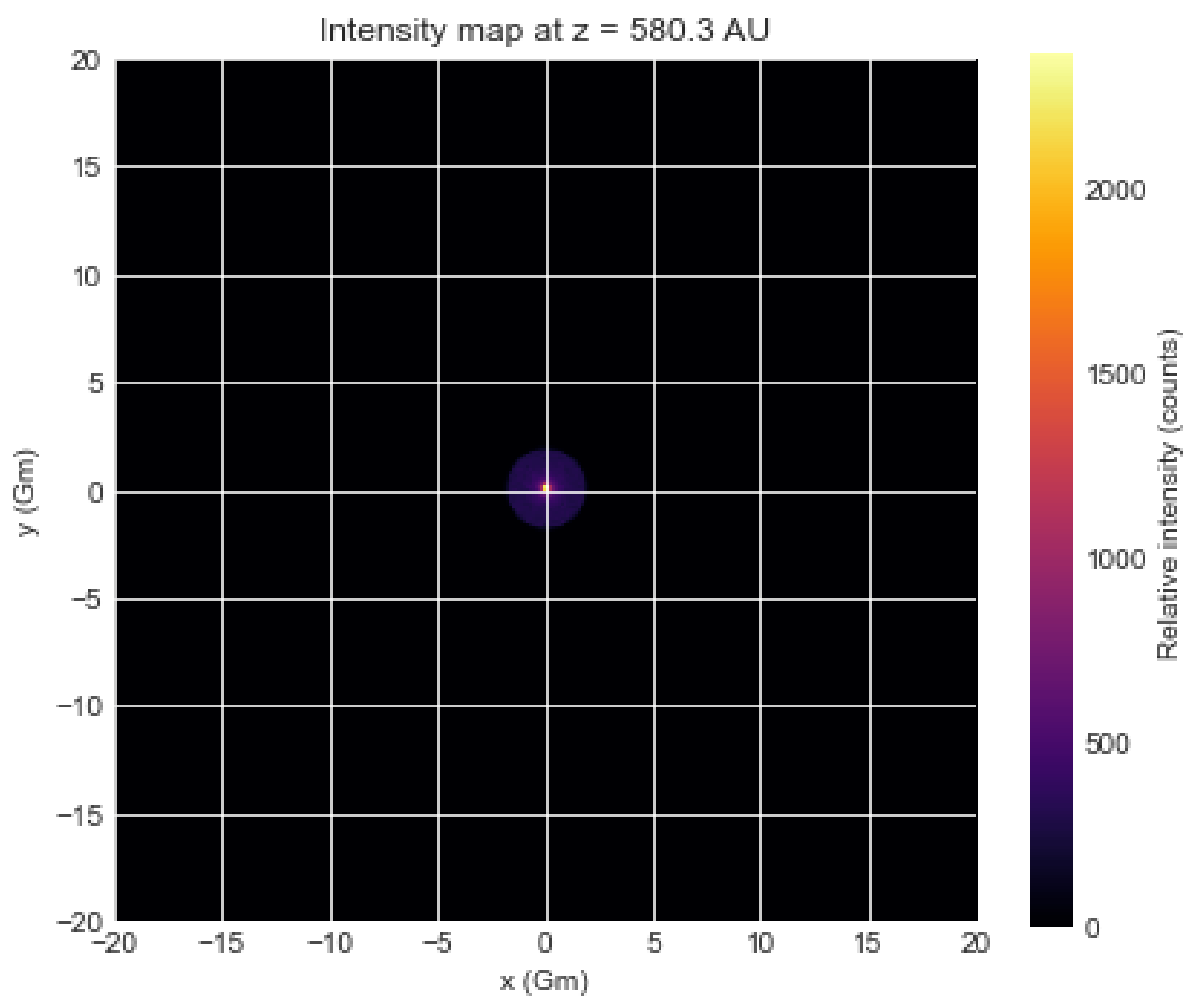
- Hence this is either for us having sent a receiver to 550AU OR an alien at 550AU OR 2 alien civilisations, one of which at 550AU (where it would be a focal line not point).
- Distances >550AU possible, but further away = longer = loss of power (by inverse square), loss of quality etc
- This code is finding the optimal distance, by finding peak intensity ... should be near 550
- Source assumed to be very far away to allow for parallel rays incoming (i.e. basically infinity)
- Tolerance in the code is 0.1AU so this is the error I think?
- Assumes sender receiver and sun on same axis
- Earth to sun is 1AU... not as far as we would like for the parallel approx. but if narrow beam its alright... better to assume everything further away
- Lowering max impact parameter again so beams all closer to sun to feel its effect. Trying 5:

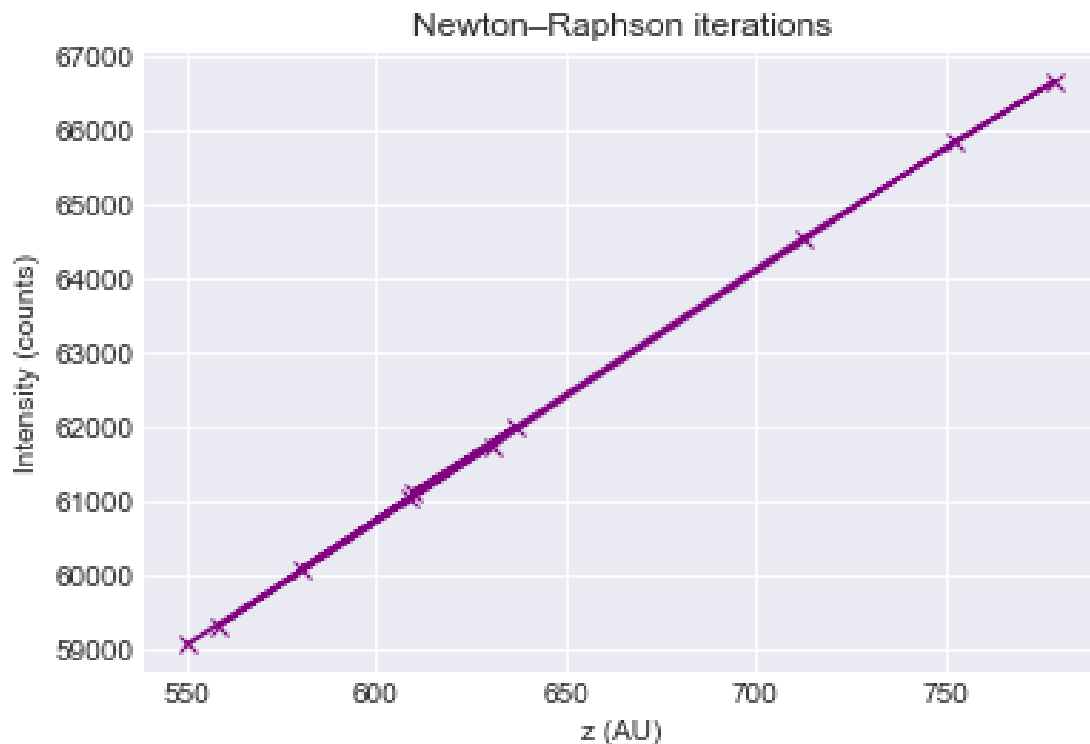


-
- Trying 10 R_{sun}



Trying 3 R_{sun} *** this ones pretty good maybe.





Rayangle() works out the angle by which the ray bends using equation $\frac{4GM}{c^2u}$

Rays() generates N random rays (Monte Carlo?)

$R = u_{\max} * \text{random.rand}(N)$ – this is a uniform dis over (0,1)

$\Phi = 2 * \pi * \text{random.rand} * N$ (- random angle and uniform

$X, y = r \cos / \sin \phi$, $u = r$

Call rayangle(u)

Small angle approx, angle in -x/y directions

$\theta_x = -\text{angle} * (x_0 / u)$

$\theta_y = -\text{angle} * (y_0 / u)$

follows rays from $z=0$ to observer

$x = x_0 + \theta_x * z_{\text{obs}}$ (returns)

$y = y_0 + \theta_y * z_{\text{obs}}$ (returns)

intensity()

calls rays() and gets x,y (array of where rays hit observer plane)

$r = \sqrt{x^2 + y^2}$ (standard)

I = sum of rays where $r < \text{aperture of receiver (chosen)}$, returns

Intensity_avg() calls intensity() an amount of times and averages the result. Could get an error in here?.

Newtonraphson()

Z=z0 initial guess, and empty list of history of the run

For attempts in range of maxattempt, uses central difference formulae after saying $dl(z)/dz = 0$ is the initial equation, since finding maximum intensity.

$dl/dz \rightarrow (I(z+dz) - I(z-dz)) / (2 dz)$

$d^2I/dz^2 \rightarrow (I(z+dz) - 2I(z) + I(z-dz)) / dz^2$

$z_new = z - (dl/dz) / (d^2I/dz^2)$

the different I havues are found by calling intensity_avg()

adds results to history list

if one of the divisors is 0, breaks so no divide by 0 (but this makes the code go crazy wrong – only happens randomly though)

if in tolerance (0.1AU here) – as in different between z and znew is lower than tolerance, breaks because we have a good result.

Z=znew, returns z and history

Intensitymap() makes a 2D histogram (np function) given the x,y of where the rays end up, and *****call this earlier to make it the map of the actual rays used in calculation surely...

Actual code

```
zopt = 0
```

```
z0guess = 550 * AU
```

```
dz = 10*AU
```

```
tol = 0.1*AU
```

```
maxattempt = 10
```

```
N = 150000
```

```
umax = 3*Rsun # originally had this as 2, but if u increaqse it it  
gets more consistent results
```

```
aperture = 1e9
```

```
zopt, history =
```

```
newtonraphson(z0guess,dz,tol,maxattempt,N,umax,aperture)
```

```
z = np.array([a[0] for a in history]) / AU
```

```
I = np.array([a[1] for a in history])
```

Plot NR iterations

```
finalz = zopt
```

```
xmap, ymap = rays(N, umax, finalz)
```

```
bins = 250
```

```
extent = 2e10
```

intensitymap(xmap, ymap, bins, extent, finalz)

NEW UPDATE. Code was terminating before netwon Raphson was finished bc of the max attempts – changed max attempts to 30. Also umax is officially 3 R_{sun} bc that feels more reasonable than 30. Also also have changed the dz value from 10Au to 1Au bc I think when it was going to the extremes its bc in one jump it would get too far away and ‘fall off’ either end basically lol, anyway not it gets close to 550 pretty much every time woohoo

